

Documentation

في مشاريع Cloud و DevOps

Practical Guide & Professional Reference

BUILD
Skills

REAL-WORLD
Examples

BEST
Practices

CAREER
Insights

Prepared by

Emad Adel

Multi-Cloud Architect | Microsoft MVP | Microsoft Certified Trainer
+18 years in IT

WEB www.emadadel.com

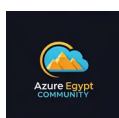
IN [linkedin.com/in/emadadel](https://www.linkedin.com/in/emadadel)

✉ me@emadadel.com

YT [@EmadAdel2008](https://www.youtube.com/@EmadAdel2008)

GH github.com/emadadel2008

✉ community@egyptazure.org



دليل عملي لتقليل وقت ال Troubleshooting وتسهيل ال Deployment وتسليم المشروع

ال Documentation مش خطوة إدارية بتتعمل بعد انتهاء المشروع. هي جزء من ال Solution نفسها، لأنها بتحفظ قرارات ال Architecture، وال Configurations المهمة، وطريقة ال Deployment وال Rollback، ويتمنع معرفة المشروع من إنها تفضل مرتبطة بشخص واحد. الدليل ده بيوضح إيه اللي محتاج يتوثق، وإزاي تخلي التوثيق جزء طبيعي من شغل الفريق.

1. تحديد المخاطر الناتجة عن وجود تفاصيل المشروع في دماغ شخص واحد.
2. فهم تأثير غياب ال Documentation على ال Troubleshooting وال Deployment وتسليم المشروع.
3. تحديد الحد الأدنى من المعلومات اللي لازم تتوثق في أي Cloud Project.
4. تحويل تحديث ال Documentation إلى خطوة ثابتة داخل Pull Request أو Deployment.
5. التمييز بين دور Infrastructure as Code ودور توثيق القرارات وطريقة التشغيل.
6. تقييم مدى جاهزية المشروع إن شخص أو فريق جديد يستلمه ويشغله.

ال Documentation جزء من ال Solution

توضيح إن التوثيق أصل هندسي وتشغيلي، مش مجرد مهمة إدارية منفصلة عن التنفيذ.

اختبار بسيط يكشف حالة المشروع

اسأل السؤال ده: لو الشخص المسؤول عن المشروع مش موجود بكرة، هل باقي الفريق هيعرف يعمل ال Deployment. يرجع إصدار سابق، أو يحل مشكلة من غير ما يرجعه؟ لو الإجابة لأ، فالمشكلة مش بس في توزيع المعرفة؛ المشكلة إن تشغيل ال Solution نفسه معتمد على شخص واحد.

- ال Documentation بتحفظ سياق القرارات، مش بس شكل البيئة الحالي.
- بتساعد الفريق يفهم إيه ال Architecture اتبنت بالطريقة دي.
- بتوضح خطوات التشغيل والتغيير والرجوع عن التغيير.
- بتقلل الوقت المطلوب لإعادة اكتشاف معلومات كانت معروفة وقت التنفيذ.
- بتسهل تسليم المشروع لفريق ثاني أو الرجوع له بعد فترة.

Documentation الجيدة مش مجرد ملفات موجودة. لازم تكون واضحة، محدثة، سهلة الوصول، وتساعد شخص تاني يشغل المشروع ويدعمه.

إزاي معرفة المشروع بتضيع؟

شرح النمط الشائع اللي بيحوّل التنفيذ السريع إلى اعتماد خطير على معرفة فردية.

التنفيذ السريع من غير تسجيل السياق

فرق كتير بتركّز على إن الـ Deployment يخلص أو المشكلة تتحل بسرعة، لكن من غير تسجيل القرار اللي اتأخذ وسببه. النتيجة إن معلومات أساسية زي Architecture decisions والـ Configurations والـ Dependencies وخطوات الـ Rollback تفضل في دماغ شخص واحد.

المعلومة اللي ضاعت	السؤال اللي بيظهر وقت المشكلة	التأثير العملي
Architecture decision	ليه اخترنا التصميم ده؟	الفريق يعيد تحليل قرار سبق اتخاذه.
Configuration change	الإعداد ده اتغير إمتى وليه؟	تشخيص أبطأ واحتمال تعديل غير مناسب.
Dependency	المكوّن ده بيعتمد على إيه؟	صعوبة فهم تأثير أي تغيير.
Deployment steps	إيه الترتيب الصحيح للتنفيذ؟	أخطاء أكثر أثناء الـ Deployment.
Rollback steps	إزاي نرجع للوضع السابق؟	وقت أطول لاستعادة التشغيل.
Known issue	هل المشكلة دي حصلت قبل كده؟	إعادة نفس خطوات البحث من البداية.

حتى لو صاحب القرار موجود حاليًا، الاعتماد عليه في كل سؤال بيعمل نقطة اختناق وبيصعب دعم المشروع بعد فترة.

التكلفة الحقيقية لغياب الـ Documentation

ربط نقص التوثيق بنتائج تشغيلية واضحة بدل التعامل مع الموضوع كمسألة تنظيمية فقط.

المشكلة مش في الكتابة! المشكلة في الوقت اللي بيضيع بعد كده

لما تفاصيل المشروع ما تكونش موثقة، الفريق بيستهلك وقت في فهم الوضع الحالي قبل ما يبدأ يحل المشكلة نفسها. وكل تغيير غير موثق بيضيف طبقة جديدة من الغموض.

الموقف	من غير Documentation كافية	مع Documentation واضحة
Troubleshooting	إعادة اكتشاف التصميم والتغييرات السابقة.	البدء من سياق واضح للقرار والإعدادات المعروفة.
Deployment	الاعتماد على الذاكرة أو خطوات غير مكتملة.	اتباع خطوات معروفة وواضحة.
Rollback	محاولة تحديد طريقة الرجوع أثناء المشكلة.	وجود خطوات رجوع موثقة مسبقًا.
تسليم المشروع	نقل المعرفة شفهيًا والاعتماد على أشخاص محددين.	وجود مرجع يشرح التصميم والتشغيل.
الرجوع للمشروع بعد شهور	فهم المشروع من الصفر تقريبًا.	استعادة السياق والقرارات بشكل أسرع.

معرفة إن Configuration معينة موجودة مش كفاية دائمًا. الفريق محتاج يعرف سببها، وتأثيرها، وإيه المطلوب لو احتاج يغيرها أو يرجع عنها.

مثال عملي: تغيير Firewall rule في Microsoft Azure

تحويل الفكرة إلى سيناريو واقعي يوضح الفرق بين تسجيل التغيير وتسجيل سياقه.

التغيير ممكن ينجح دلوقتي ويعمل مشكلة بعدين |

تخيل إن حد غيّر Firewall rule في Microsoft Azure لحل احتياج معين، لكنه ما سجلش سبب التغيير ولا القيمة القديمة. لو ظهرت مشكلة بعد كده، الفريق مش هيبداً بإصلاحها مباشرة؛ الأول هيحاول يفهم إيه اللي اتغيّر عليه، وهل الرجوع عن التغيير آمن ولا لا.

العنصر	لو مش موثق	المعلومة المطلوبة
سبب التغيير	الفريق مش هيعرف المشكلة الأصلية اللي كان التغيير بيعالجها.	ليه الـ Firewall rule اتغيّرت؟
الحالة السابقة	الرجوع للوضع القديم هيبقى غير واضح.	إيه القيمة أو الحالة القديمة؟
السياق التشغيلي	ممكن يتشال التغيير من غير فهم تأثيره.	إيه الجزء اللي بيعتمد على التغيير؟
طريقة الرجوع	الفريق هيجدد الـ Rollback تحت ضغط المشكلة.	إزاي نرجع بأمان لو التغيير سبب مشكلة؟

تسجيل إن الـ Firewall rule اتغيّرت مفيد، لكن القيمة الحقيقية بتيجي من تسجيل سبب التغيير، والحالة السابقة، وطريقة الـ Rollback.

الحد الأدنى المطلوب في Documentation المشروع |

تقديم Checklist عملية تغطي المعلومات الأساسية من غير تحويل التوثيق إلى عبء كبير.

إيه اللي لازم يتوثق؟ |

الفائدة	المحتوى المطلوب	جزء الـ Documentation
يساعد الفريق يفهم شكل الـ Solution بسرعة.	رسم بسيط يوضح المكونات الرئيسية والعلاقات بينها.	Architecture Diagram
يمنع إعادة مناقشة القرار من غير سياقه.	القرارات المهمة وأسباب اختيارها.	Architecture decisions
يسرّع فهم الوضع الحالي وتشخيص المشكلات.	الإعدادات المهمة التي تشغيل المشروع ويعتمد عليها.	Configurations
يوضح تأثير التغيير ونقاط الترابط.	الأنظمة أو المكونات التي المشروع يعتمد عليها.	Dependencies
يقلل الاعتماد على الذاكرة والخطوات الشفهية.	الخطوات المطلوبة لتنفيذ الـ Deployment.	Deployment
يساعد الفريق يتعامل مع فشل التغيير بشكل أوضح.	الخطوات المطلوبة للرجوع عن الـ Deployment أو التغيير.	Rollback
يمنع تكرار نفس رحلة الـ Troubleshooting.	المشاكل المعروفة وطريقة التعامل معها.	Known issues

وثق الإعدادات وطريقة استخدامها، لكن ما تحطش أي Secrets داخل ملفات الـ Documentation أو الـ Repository.

مش لازم تبدأ بمستند ضخمة. Architecture Diagram بسيطة، وأهم القرارات، وخطوات Deployment و Rollback واضحة ممكن تمنع وقت كبير من إعادة الاكتشاف.

خلي الـ Documentation جزء من طريقة الشغل

توضيح إزاي يفضل التوثيق محدثًا عن طريق ربطه بالتغييرات الفعلية داخل المشروع.

التوثيق بعد انتهاء المشروع غالبًا بيتأجل

لو تحديث الـ Documentation مهمة منفصلة بتتعمل بعد التنفيذ، غالبًا هتتأجل أو هتتعمل بشكل ناقص. القاعدة العملية هي إن أي تغيير مؤثر يراجع معاه التوثيق المرتبط به.

- خلي مراجعة الـ Documentation جزء من كل Pull Request له تأثير على التصميم أو التشغيل.
- راجع خطوات الـ Deployment والـ Rollback عند تنفيذ تغيير مرتبط بيها.
- سجّل سبب التغيير، مش التغيير نفسه بس.
- خزّن الـ Documentation بصيغة Markdown داخل الـ Repository جنب المشروع.
- حدّث الملفات في نفس دورة التغيير عشان الكود والتوثيق ما ينفصلوش عن بعض.

نوع التغيير	سؤال المراجعة قبل الإغلاق
تغيير في Architecture	هل الـ Architecture Diagram والقرار وأسبابه اتحدّثوا؟
تغيير Configuration	هل سبب التغيير والحالة السابقة موثّقين؟
تغيير Deployment	هل خطوات الـ Deployment الحالية ما زالت صحيحة؟
إضافة Dependency	هل الاعتماد الجديد وتأثيره موضحين؟
إضافة Rollback أو تعديله	هل خطوات الرجوع واضحة ومحدّثة؟
حل مشكلة متكررة	هل اتضافت ضمن Known issues وطريقة التعامل معاها؟

قبل ما تقفل Pull Request أو Deployment، اسأل: هل التغيير ده خلّى أي جزء من الـ Documentation غير صحيح أو ناقص؟

Infrastructure as Code مش بديل كامل

تحديد حدود Terraform و Infrastructure as Code وتوضيح المعلومات اللي ما زالت محتاجة توثيق بشري واضح.

إيه اللي Terraform بيوثقه وإيه اللي ما بيوثقوش؟

استخدام Terraform أو أي Infrastructure as Code يساعد في تمثيل جزء من الـ Infrastructure. لكن وجود الملفات مش معناه إن كل معرفة المشروع بقت موثقة.

نوع المعلومة	دور Infrastructure as Code	هل محتاجة Documentation إضافية؟
تعريف جزء من الـ Infrastructure	ييمثل الموارد والإعدادات الموجودة داخل التعريفات.	ممكن تحتاج شرح للسياق وطريقة التشغيل.
سبب اختيار Architecture معينة	مش بيشرح القرار بالضرورة.	نعم، لازم يتوثق السبب.
سبب تغيير Configuration	وجود التغيير لا يشرح الدافع وراءه.	نعم، لازم يتوثق السياق.
طريقة التشغيل والتسليم	مش بديل عن خطوات التشغيل الواضحة.	نعم، لازم تتوثق.
Known issues وطريقة التعامل معاها	مش الهدف الأساسي من الـ Infrastructure as Code.	نعم، لازم تتوثق.

Infrastructure as Code بيوثق جزء من حالة الـ Infrastructure، والـ Documentation بتكمل الصورة بشرح القرارات والسياق وطريقة التشغيل والـ Rollback.

مراجعة جاهزية المشروع للتسليم والدعم

توفير اختبار نهائي يساعد الفريق يحدد إذا كانت الـ Documentation قابلة للاستخدام فعلاً.

Checklist قبل إغلاق الـ Task أو تسليم المشروع

- هل فيه Architecture Diagram بسيطة ومفهومة؟
- هل القرارات المهمة وأسبابها مسجلة؟
- هل الـ Configurations والـ Dependencies المهمة واضحة؟
- هل خطوات الـ Deployment مكتوبة بترتيب مفهوم؟
- هل خطوات الـ Rollback موجودة وواضحة؟
- هل Known issues وطريقة التعامل معاها مسجلة؟
- هل أي تغيير حديث اتوثق ضمن نفس Pull Request أو Deployment؟
- هل الملفات موجودة في مكان سهل الوصول، زي الـ Repository جنب المشروع؟
- هل تم التأكد إن الملفات ما فيها Secrets؟
- هل شخص تاني يقدر يستلم المشروع ويشغله ويدعمه؟

لو شخص تاني استلم الشغل بكرة، هل هيعرف يعمل Deployment، يتعامل مع مشكلة، ويرجع عن تغيير من غير ما يبدأ فهم المشروع من الصفر؟

الملخص

الـ Documentation الجيدة بتحمي معرفة المشروع من الضياع، وتقلل وقت إعادة اكتشاف القرارات أثناء الـ Troubleshooting، وتسهل الـ Deployment والـ Rollback وتسليم الشغل. ابدأ بالحد الأدنى المفيد، خزنه بصيغة Markdown داخل الـ Repository، وحدّثه مع كل Pull Request أو Deployment مؤثر. ولو بتستخدم Terraform أو Infrastructure as Code، اعتبره جزءاً من التوثيق، مش بديلاً عن شرح القرارات والسياق وطريقة التشغيل.